

CINEMATCH

Presentation Transcript

Advanced C Programming — Capstone Project
Sejong University · 2024

Member	Topic	Slides	Time
Member 1	Introduction, Problem & Solution	1 – 3	2:30
Member 2	Features & Functionalities	4 – 5	2:30
Member 3	Architecture & C Engine	6 – 8	2:30
Member 4	AI Integration & External APIs	9 – 11	2:30
Member 5	Backend Structure & Authentication	12 – 13	2:30
Member 6	Data Flow, Testing & Conclusion	14 – 17	2:30

Total presentation time: 15 minutes

MEMBER 1

Introduction, Problem & Solution · Slides 1 – 3 · ≈ 2:30

[Slide 1 — Title Slide]

Good morning everyone. My name is [Name], and on behalf of our team, welcome to the presentation of CINEMATCH — a mood-based movie and game recommender we built as our capstone project for Advanced C Programming here at Sejong University.

Our team of six has spent this semester combining artificial intelligence, C programming, and modern web development to create something that solves a real everyday problem. Let me start by telling you what that problem is.

★ **Speaker tip:** *Pause one second after the welcome sentence. Make eye contact with the audience before moving on.*

[Slide 2 — The Problem]

Here is a question: when was the last time you spent more than 15 minutes deciding what to watch — and ended up watching nothing?

This is called decision fatigue. Netflix alone has over 15,000 titles available. YouTube has billions of videos. Steam has over 50,000 games. Having too many choices without a filter that understands our current state of mind actually makes it harder to decide — not easier.

Current recommender systems try to help by using your viewing history and popularity rankings. But they only know your past. They do not know that tonight you are tired and want something light. They do not know you are watching with three friends who hate horror. They know what you watched last week — not what you need right now.

This gap — between what existing platforms know and what users actually need — is the exact problem CINEMATCH is built to solve.

★ **Speaker tip:** *Slow down on 'They know your past — not your present.' This is your key contrast line.*

[Slide 3 — Our Solution]

CINEMATCH gives you one simple interaction: you describe your mood in plain words, and the system finds your perfect match.

You might type: 'something scary but short to watch with friends tonight.' And within seconds you get a recommendation — with a clear, personalized explanation of exactly why that title fits you right now.

The pipeline behind this has three layers. First, an AI model reads your natural language description and converts it into five structured preferences: medium, genre, time commitment, emotional tone, and social context. Second, a scoring engine we wrote in C takes those five values and runs a weighted algorithm across our library of 55 curated titles, calculating a numerical score for each one. Third, the AI takes the highest-scoring result and writes a two-sentence explanation tailored to your original mood.

The intelligence of the AI and the precision of the C algorithm work together — neither one alone would give you this result.

I'll now hand over to [Member 2], who will walk you through every feature we built.

★ **Speaker tip:** *Point at the three-step diagram on screen as you say 'three layers.' Slow down for the handoff.*

MEMBER 2

Features & Functionalities · Slides 4 – 5 · ≈ 2:30

[Slide 4 — Features Overview]

Thank you [Member 1]. My name is [Name], and I will walk you through all five features of CINEMATCH.

CINEMATCH is not just a recommendation engine — it is a complete entertainment decision platform. We built five distinct pages, each solving a different part of the problem.

The first feature is AI Match. This is the core of our project. You type a mood description in plain English, hit the button, and the AI-plus-C-engine pipeline runs. You receive the recommended title, its genre and rating, and a custom explanation of why it fits your current mood. If you are logged in, the recommendation is automatically saved to your history.

The second feature is the Movies Browser, powered by TMDB — The Movie Database. This is a real open database with hundreds of thousands of films. You can search by title, filter by genre — Action,

Comedy, Drama, Horror, Sci-Fi, and more — and browse a poster grid. Clicking any movie opens a detailed view with cast, rating, and overview.

The third feature is the Games Browser, powered by RAWG — one of the world's largest game databases with over 800,000 titles. Same interface: search, filter, browse covers, open details.

The fourth feature is the Discussion Page. You search any movie — not just the 55 in our library, but any film on TMDb. The system fetches the cast with photos and backdrop, and our AI generates a deep analysis: the main theme, the epic moments, the director's vision, and discussion questions you could use with friends.

The fifth feature is User Accounts. You register with a username and password, log in from any browser session, and your complete recommendation history is saved — including the date, your original mood text, and the AI explanation. Passwords are hashed — they are never stored in plain text.

Together, these five features take CINEMATCH from a simple class project to a fully functional web application.

I'll now hand over to [Member 3], who will explain the technical architecture.

★ **Speaker tip:** *Keep a steady pace — you have the most content. Use the slide visuals to anchor each feature.*

MEMBER 3

System Architecture & C Engine · Slides 6 – 8 · ≈ 2:30

[Slide 6 — System Architecture (DDD)]

Thank you [Member 2]. My name is [Name], and I will explain how CINEMATCH is structured technically — starting with the overall architecture.

We designed the entire system using a pattern called Domain-Driven Design, or DDD. The idea is to keep the core business logic completely separate from technical details like the web server or the AI API.

The code is split into four layers. The Domain layer contains the core data structures and rules — what a Title is, what Preferences look like. Nothing in this layer knows about HTTP or AI. The

Application layer contains the use cases — the steps that orchestrate the full pipeline. The Infrastructure layer is where the outside world connects: the OpenRouter AI client and the runner that calls our C executable. And the Interface layer is the HTTP API and the React frontend.

This separation means each layer can be changed without breaking the others. It also made it much easier to write and test each part independently.

★ **Speaker tip:** *Point to each layer on the architecture diagram as you name it.*

[Slide 7 — C Engine: Struct & Scoring Algorithm]

The heart of our project is the C engine, located in the engine/ folder.

In titles.h, we define a Title struct with eight fields: type, title, year, genre, mood, timeNeed, social, rating, and description. We then declare a global array called library containing all 55 Title objects. This is our curated dataset — every title hand-picked for genre, mood, time, and social context coverage.

The scoring algorithm lives in recommend.c, inside the scoreTitle function. It receives one Title and the user's five preference values, and returns a floating-point score. The scoring rules are: genre match gives 6 points — the strongest signal. Medium match gives 5 points. Mood match gives 4. An exact time length match gives 3 points; a neighbouring length gives 1. Social context match gives 2 points. Finally, the title's rating is divided by 10 and added as a tiebreaker — so an 8.7-rated film adds 0.87 to its total score.

The function pickBest loops through all 55 titles, skips any that do not match the requested medium, and tracks the highest scorer. The result is the best title for that mood.

★ **Speaker tip:** *The scoring weights table is on screen. Walk through each row slowly — this is the C code explanation the audience needs to follow.*

[Slide 8 — Engine Mode: How Node Calls the C Program]

The C engine runs in two modes.

In interactive mode — when you run it from the terminal with no arguments — it asks you five multiple-choice questions and prints a human-readable result. This is how we tested it during development.

In engine mode — when the Node.js backend calls it — it passes five command-line arguments: medium, genre, timeNeed, mood, and social. The engine detects argc equals 6, skips the quiz, runs

the scoring, and prints one line of JSON to standard output. For example: open brace, found: true, type: Movie, title: Interstellar, year: 2016, rating: 8.6, close brace.

The Node backend calls this executable using Node's `child_process` module, captures the standard output, and parses the JSON. This is a clean, language-agnostic interface between C and JavaScript.

I'll now pass to [Member 4], who will explain the AI integration and external APIs.

★ **Speaker tip:** *If anyone has studied operating systems — mention that this is a classic UNIX pipe pattern: parent process reads child stdout.*

MEMBER 4

AI Integration & External APIs · Slides 9 – 11 · ≈ 2:30

[Slide 9 — AI Integration: OpenRouter]

Thank you [Member 3]. My name is [Name], and I will explain how the AI layer works and what external APIs power our application.

The AI component uses a service called OpenRouter. OpenRouter is a gateway that gives you access to dozens of large language models through a single API endpoint — the same format as OpenAI, so switching models requires only changing one string in the code.

We use a three-model fallback chain to ensure reliability. The primary model is Qwen3 235B — a very capable open-source model that is free under the rate limit. If that returns a rate-limit error or server error, we automatically retry with Llama 3.3 70B. If that also fails, we fall back to a 120-billion parameter open model. If all three fail — which is rare — the system falls back to keyword matching to guarantee the user always gets a result.

This fallback chain means the application is resilient. During demos, the AI has always returned a result within 10 seconds.

★ **Speaker tip:** *Emphasize 'always returns a result' — reliability is a key engineering concern the professor will appreciate.*

[Slide 10 — AI Pipeline: Mood → Preferences → Explanation]

The AI performs two separate tasks in our pipeline.

The first task is mood-to-preferences. The backend sends the user's natural language text to the AI with a strict system prompt that instructs it to return only a JSON object with five fields: medium, genre, timeNeed, mood, and social. For example, if a user types 'something scary but short to watch with friends tonight', the AI returns: genre colon horror, timeNeed colon short, social colon group, mood colon intense. The C engine then uses exactly these five values as input.

The second task is explanation. After the C engine picks the best title, the backend sends the title details back to the AI and asks it to write a two-sentence explanation of why this specific title fits the user's original mood. This is what gives CINEMATCH its personalized feel — the explanation is never generic, it always references the specific mood the user described.

Both AI calls use the same OpenRouter client, the same fallback chain, and the same JSON extraction logic.

★ **Speaker tip:** *If you have time, read one example explanation out loud — it makes the AI feel real and impressive.*

[Slide 11 — External APIs: TMDB and RAWG]

For the browse and discussion features, we use two external databases.

TMDB — The Movie Database — is a free, community-maintained database of hundreds of thousands of films and TV shows. We use it in three places: the Movies browser for search and poster grids, the Discussion page for movie search, cast photos, and backdrop images, and as the source of the movie overview we send to the AI for discussion analysis. Poster images are fetched at w342 resolution from TMDB's image CDN.

RAWG is one of the world's largest video game databases, with over 800,000 titles. We use it for the Games browser — search, genre filtering, cover images, and game details.

All three API keys — OpenRouter, TMDB, and RAWG — are stored in environment files that are listed in our .gitignore. They are never committed to GitHub. This is a security practice we enforced from the very first commit.

I'll now pass to [Member 5], who will explain the backend structure.

★ **Speaker tip:** *Mention that TMDB and RAWG are both free-tier APIs — this is relevant because it means CINEMATCH can run with zero hosting cost.*

MEMBER 5

Backend Structure & Authentication · Slides 12 – 13 · ≈ 2:30

[Slide 12 — Backend: Express Server & API Endpoints]

Thank you [Member 4]. My name is [Name], and I will explain the backend — how the Node.js server is organized and what endpoints it exposes.

The backend is a Node.js server using the Express framework, running on port 3000. Following the Domain-Driven Design structure [Member 3] described, the server code is organized into domain, application, infrastructure, and interfaces folders.

We expose seven API endpoints. POST slash api slash recommend is the main pipeline endpoint — it receives the mood text, runs the AI mood analysis, calls the C engine, runs the AI explanation, and returns the full result. POST slash api slash discuss triggers the AI-powered movie discussion analysis. GET slash api slash titles returns the full 55-title curated library as JSON. POST slash api slash auth slash register and POST slash api slash auth slash login handle user account creation and login. GET slash api slash history and POST slash api slash history load and save a user's recommendation history — these two endpoints require a valid authentication token.

The DDD separation means each endpoint is a thin HTTP wrapper. It validates the request, calls the relevant application-layer function, and returns the result. No business logic lives in the HTTP layer itself.

★ **Speaker tip:** *Point to the endpoint list on screen as you name each one — don't try to memorize them all, just read them.*

[Slide 13 — User Authentication: bcrypt + JWT]

Now let me explain how user authentication works — this was one of the most technically interesting parts we built.

When a user registers, we never store their password directly. Instead, we use a library called bcryptjs to hash it with 10 salt rounds. Bcrypt is a one-way hashing function — this means no one, including us, can ever recover the original password from the stored hash. This is the industry standard approach used by real production systems.

When the user logs in, bcrypt compares their typed password against the stored hash. If it matches, we generate a JSON Web Token — a JWT — using the jsonwebtoken library. This token is signed with a secret key, contains the user's ID and username, and has a 7-day expiry. The frontend stores

this token in the browser's localStorage and sends it in the Authorization header on every request to a protected endpoint.

User data is stored in a JSON file at server slash data slash users.json. This file is gitignored — it never appears in our GitHub repository. Each user record contains their ID, username, hashed password, creation timestamp, and recommendation history — up to 50 entries, newest first.

The reason we chose a JSON file over a database is simplicity — no database server to install, no native modules to compile, and the project runs on any machine with just Node.js installed.

I'll now pass to [Member 6], who will trace the complete data flow and close the presentation.

★ **Speaker tip:** *The words 'no one can recover the original password' always land well with a general audience. Pause after saying it.*

MEMBER 6

Data Flow, Testing & Conclusion · Slides 14 – 17 · ≈ 2:30

[Slide 14 — Complete Data Flow (End-to-End)]

Thank you [Member 5]. My name is [Name], and I will trace a single request all the way through CINEMATX, then cover our testing and close the presentation.

Let me walk through exactly what happens when you type 'scary short film with friends' and press the button.

Step one: the React frontend sends a POST request to slash api slash recommend with the mood text in the request body.

Step two: the Express server calls moodToPreferences, which sends the text to OpenRouter. The AI responds in under 3 seconds: genre horror, timeNeed short, social group, mood intense.

Step three: the backend spawns the C executable as a child process, passing those five values as command-line arguments. The C engine scores all 55 titles and prints one line of JSON to standard output. Node captures that output and parses it — result: Alien, 1979, rating 8.5.

Step four: the backend calls `explainRecommendation`, sending the title details back to the AI. The AI writes: 'Alien is a perfect short-and-sharp horror experience for a group night — its claustrophobic tension builds fast and hits hard.' Node receives this and packages the full response.

Step five: the complete JSON result is sent back to the frontend and rendered — title card, genre, rating, and the personalized explanation.

If the user is logged in, the frontend simultaneously fires a POST to `/api/history` to save the result. Total round-trip: typically 6 to 10 seconds on the free AI tier.

★ **Speaker tip:** *Speak slowly on each step number. Pausing between steps helps the audience follow the flow.*

[Slide 15 — Testing & Evaluation]

For testing, we ran the full pipeline with four different mood inputs and verified the outputs matched our expectations.

'Relaxing cozy solo night' correctly returned *Balatro* — a calm, solo strategy game. 'Scary horror with a group' returned *Alien* — a classic group horror film. 'Epic action game for hours' returned *Super Mario Odyssey*. And 'Thoughtful sci-fi alone' returned *Arrival* — a deep, solo science fiction film.

All four test cases passed with the exact expected output.

We also tested four edge cases. A mixed mood like 'funny but also scary' — the C engine selects the genre with the higher scoring weight. An AI failure — the system falls back to keyword matching and still returns a result. An empty input — the server returns a 400 error and the frontend shows a clear message. And pure gibberish — the AI still produces a preference set, and the engine picks the closest match.

In all cases, the system returned a valid, usable result.

[Slide 16 — Limitations & Future Work]

We are honest about the current limitations of CINEMATCH.

The AI Match feature is limited to 55 curated titles — the browse and discussion pages use open databases, but the C engine recommendation only covers our library. The application currently runs locally and is not deployed on a public server. And free AI models can be slow during peak hours.

For future work, we have identified five improvements. First, user profiles with watch history and preferences that improve over time. Second, feedback-based personalization — if you rate a recommendation, the system learns. Third, expanding the C library to hundreds of titles. Fourth, cloud deployment so the app is accessible from any device. Fifth, automated unit tests for the C scoring logic to prevent regressions.

[Slide 17 — Thank You]

To close — CINEMATCH demonstrates that C programming is not just for systems software. Our C engine is the precise, deterministic core of a modern AI-powered web application. The mood-to-match pipeline combines the flexibility of large language models with the speed and predictability of compiled C code.

We built a working application: real user accounts, real open databases, real AI integration — from scratch, in one semester.

Thank you for your attention. We are happy to take any questions.

★ **Speaker tip:** *Smile and make eye contact as you say 'Thank you.' The whole team should stand or be visible during Q&A.*